

Министерство науки и высшего образования Российской Федерации
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Образовательная программа: Компьютерные системы и технологии



Отчет

По предмету

Программирование на C++

Лабораторная работа № 2

Студенты:

Андрейченко Леонид Вадимович

Мамонтов Иван Олегович

Преподаватель:

Лаздин Артур Вячеславович

Санкт-Петербург

2024

Задание

Необходимо спроектировать класс, реализующий хранение данных, связанных с экземпляром класса в динамической памяти. Это может быть, например представление целых чисел, для которых помимо значения типа `int` хранится строковое представление числа (22 и «двадцать два»).

Пример условен.

Для данного класса необходимо реализовать все необходимые конструкторы, включая конструкторы копирования и перемещения, деструктор. Все конструкторы и деструктор должны «сообщать» о своём вызове. `std::cout << "Copy constructor" << std::endl;` то же относится и к перегрузке операции присваивания (два варианта: без перемещения и с перемещением).

Определение класса должно быть помещено в заголовочный файл (.hpp), реализация методов в отдельном файле .cpp.

Написать программу (ещё один модуль .cpp) осуществляющую работу с экземплярами разработанного класса, которая должна демонстрировать:

- создание статических и динамических экземпляра класса, стандартного вектора в стиле Си, написать функцию для обработки данного вектора;
- передача экземпляров класса в функцию, и возврат экземпляра из функции; включая вариант с передачей и возвратом значений типа ссылки на класс;
- продемонстрировать работу с разработанным классом, создавая вектора и списки экземпляров класса, используя стандартные классы `vector` и `list` длиной от 5 до 10.

Проанализировать процессы создания и удаления экземпляров класса для различных примеров. Необходимо получить вывод от всех конструкторов, деструктора и перегруженных операций присваивания.

Выполнение

Исходники проекта доступны по ссылке:

<https://github.com/buffer404/cpp-course>

Класс хранит размерность массива и сам массив. Реализованы все базовые конструкторы, деструктор, операторы присваивания, два метода для получения размера массива и копии массива. Для проверки корректности конструкторов копирования создана функция, проверяющая соответствие экземпляров пользовательского класса

Пример вывода при выполнении:

[illegible]

```
CustomClass* a = new CustomClass;
CustomClass* b = new CustomClass[10];
delete a;
delete[] b;
```

dynamic creation:

```
Default constructor
Default constructor
Default constructor
Default constructor
Default constructor
Default constructor
Default constructor
Default constructor
Default constructor
Default constructor
Destructor
Destructor
Destructor
Destructor
Destructor
Destructor
Destructor
Destructor
Destructor
Destructor
```

```
int mas[10];
for (int i = 0; i < 10; ++i) {
    mas[i] = log(i);
}
CustomClass a(10, mas);
CustomClass b(a);
if (compareVectors(a, b)) {
    printf("vectors are equal\n");
} else {
    printf("vectors are inequal\n");
}
```

comparation:

```
Explicit constructor
Explicit constructor
Copy constructor
vectors are equal
Destructor
Destructor
```

```
printf("\nfunction exchange:\n");
CustomClass a;
acceptObject(a);
acceptRefObject(a);
getObject();
getRefObject();
```

function exchange:

```
Default constructor
Explicit constructor
Copy constructor
func got class object
Destructor
func got class object by ref
func returns class object
Explicit constructor
Destructor
func returns class object
Explicit constructor
Destructor
```

```
printf("\nvector and list creation:\n");  
std::vector<CustomClass>(10);  
std::list<CustomClass>(5);
```

```
vector and list creation:  
    Default constructor  
    Default constructor  
    Default constructor  
    Default constructor  
    Default constructor  
    Default constructor  
    Default constructor  
    Default constructor  
    Default constructor  
    Default constructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Default constructor  
    Default constructor  
    Default constructor  
    Default constructor  
    Default constructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor  
    Destructor
```

Выводы

В ходе работы был создан пользовательский класс с различными конструкторами и операторами присваивания. Была изучена последовательность вызова методов класса при создании его экземпляров.